

CRS Developer Guide

Guthrie Document Centre (CRS) for SD Guthrie

How the system is built, how to install, run, build and deploy it, which SharePoint APIs each part uses, how the configuration lists drive the code, and how the Power Automate flows work. Written for SharePoint Framework (SPFx) developers taking over the project.

Repository `sd-gathrie` · SPFx 1.23 · package version at handover 1.0.578.0 · 1 October 2026

Contents

1. Architecture	4
Technology.....	4
Upload to approval, end to end.....	5
Libraries	5
Folder tree and permissions	6
Code layout.....	6
2. What to install, and the access you need	7
Software	7
Access (ask SD Guthrie IT).....	7
3. Run it locally.....	8
4. Build	9
5. Deploy.....	10
Extensions: check they are registered	10
6. Configuration lists and how the code uses them	11
How the code finds lists.....	11
The lists	11
CRS Config settings rows	12
7. Components.....	13
Upload Form	13
Bulk Upload.....	14
Approval Document and the Approve or reject panel	15
My Submissions and Document-Viewer	16
Requests	17
Search	18
Folder Administration and Folder Reconciliation	18
Folder Structure and Migration	19
Group Management	20
Settings, Configuration and Access pages	21
Audit Log.....	21
Extensions.....	22
APIs used by each component.....	22
8. Power Automate.....	24
8.1 Where the flows live	24
8.2 If gdc's password changes: reconnect	24
8.3 Flow reference	24

8.4 Columns the flows depend on	29
8.5 Emails the system sends	30
8.6 How-tos.....	30
8.7 Rules for editing flows	32
9. Set up a test site	33
10. Troubleshooting.....	34
11. Open items at handover	35
Ready to roll out on CRS (tested on the test site)	35
Waiting on someone.....	35
Known gaps (accepted for now)	35

1. Architecture

People use SPFx web parts that talk to SharePoint as the signed-in user. Anything a user is not allowed to do themselves (write metadata, move approved files, delete, send email) is done by Power Automate flows running as the service account `gdc@sdguthrie.com`.

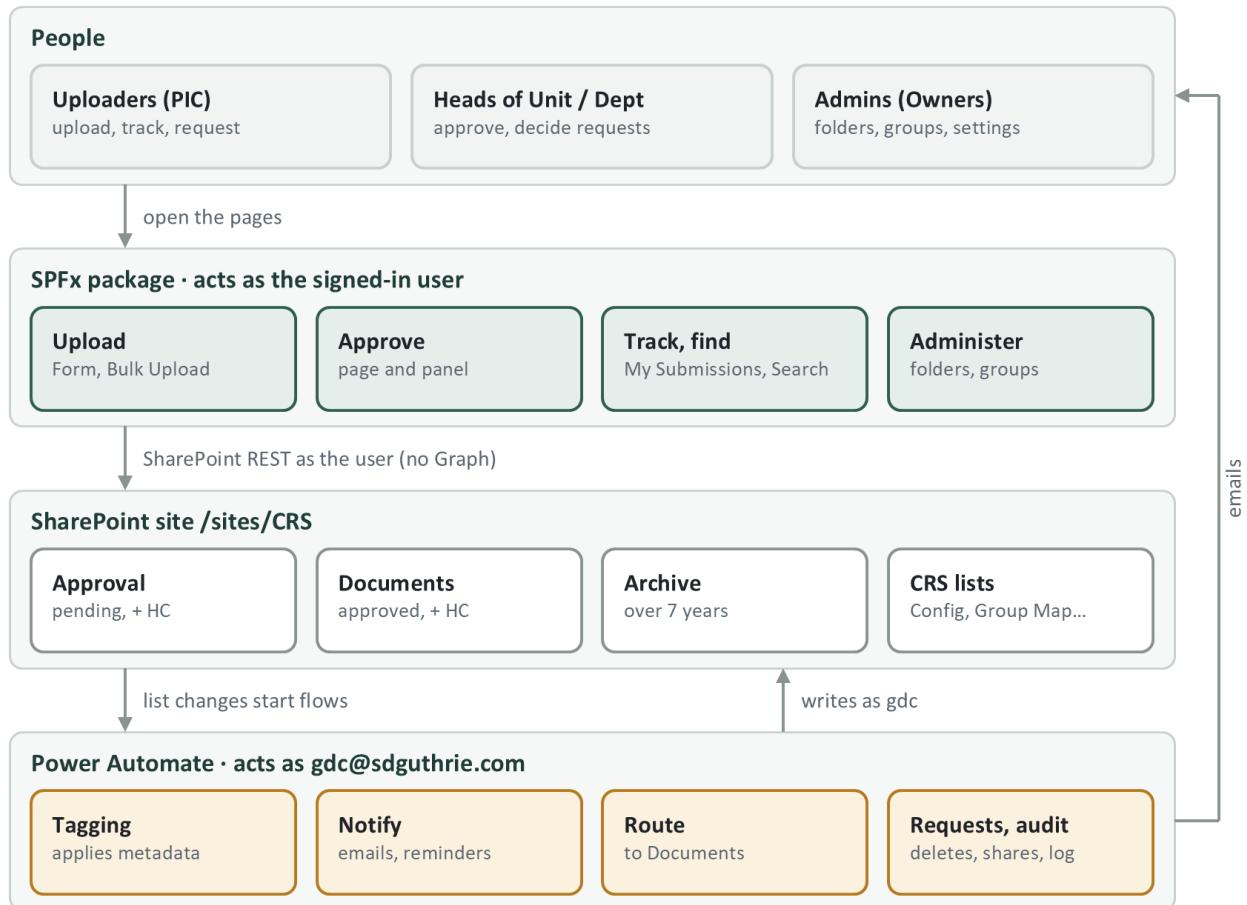


Figure 1. The four layers of CRS

Technology

- SPFx **1.23**, React 17, TypeScript 5.8, Fluent UI 8. Built with **Heft** (not Gulp).
- All data access is **SharePoint REST** through SPFx's `SPHttpClient`. No Microsoft Graph and no admin consent. (@pnp/sp is installed but not used.)
- One solution, `sd-gathrie.sppkg`: 17 web parts and 2 active extensions, deployed to the CRS site's own App Catalog.
- About 25 Power Automate flows, all running as `gdc@sdguthrie.com` (chapter 8).

Upload to approval, end to end

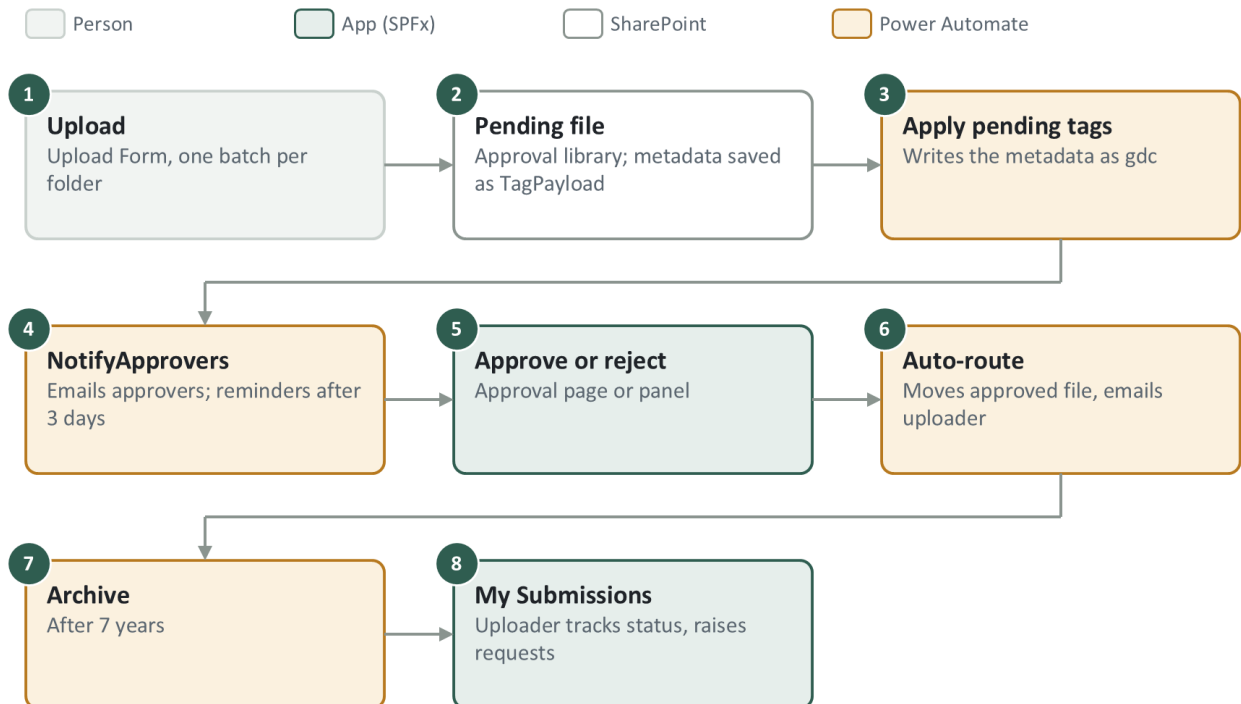


Figure 2. The path every document takes

- Uploaders cannot edit items, so the metadata goes through **CRS Submissions** (TagPayload) and the flow CRS — Apply pending tags.
- Rejected files stay in the approval library so the uploader can see the comment.
- Bulk Upload (historical files) skips approval: a flow approves them straight away.

Libraries

Six libraries in three pairs (normal and Highly Confidential, "HC"). The client renames library titles; the URL never changes.

Role	Title at handover	URL	Content approval
Pending and rejected uploads	Approval for Document	ApprovalDocument	On (approvers + author see drafts)
Approved documents	Restricted & Confidential Document	Shared Documents	Off
Pending HC uploads	Approval for Highly Confidential Document	HCApprovalDocument	On
Approved HC documents	Highly Confidential Document	HCDocuments	Off
Older than 7 years	Archive Restricted & Confidential Document	Archive	Off, read-only
HC older than 7 years	Archive Highly Confidential Document	HCArchive	Off, read-only

Folder tree and permissions

```
<Library> / <Segment> / <Department> / <Unit> / [Sub Unit] / <Year> / <Document Type> / file  
|-- permissioned ---|   |- inherit the unit's permissions --|
```

- Folders come from the site's **term store**; folder names are short codes from **CRS Term Abbreviation** (e.g. GH0/GF/GIT).
- Each unit has about five SharePoint groups (uploader, approver, viewer and HC versions), e.g. GH0_GF_GIT_APPROVER. The list of roles is PERSONAS in `src/shared/groupMapModel.ts`.
- **Folder Reconciliation** (Folder Administration page) creates the folders and grants each group its folders from **CRS Group Map**. It only adds permissions, never removes them.
- Nobody holds a delete permission. A delete is a request row that a flow carries out as gdc.
- System admins are the site's **Owners** group.

Code layout

Folder	What is in it
<code>src/webparts/</code>	One folder per web part (chapter 7).
<code>src/extensions/</code>	<code>hideAppBar</code> (+ New Folder button; hides SharePoint's own Upload, Approve/Reject and Share) and <code>bulkApprove</code> (the "Approve or reject" panel). <code>uploadCommand</code> is legacy: it links to an old sandbox site and is not registered. Ignore it.
<code>src/shared/</code>	The app's logic (permissions, folder paths, upload checks, list names). Pure logic in <code>x.ts</code> with a <code>x.test.ts</code> ; SharePoint calls in <code>spX.ts</code> . Change behaviour here and run its test.
<code>config/</code>	<code>package-solution.json</code> (version, component list), <code>serve.json</code> (local run pages).
<code>scripts/</code>	Browser-console scripts for checking live data (<code>check-*</code> , <code>diagnose-*</code> read only; <code>repair-*</code> , <code>purge-*</code> write, off by default) and two Python flow generators.
<code>PowerAutomateFlowsSDG/</code>	Exports of the live flows (25 September 2026). The source of truth for any flow question.

2. What to install, and the access you need

Software

- **Node.js 22** (22.14 or later, below 23). Use nvm (nvm-windows on Windows) to switch versions.
- Git and an editor (VS Code).
- Python 3, only if you rebuild the two generated flows (chapter 8).

```
nvm install 22
nvm use 22
npm ci
npx heft trust-dev-cert      (once per machine: trusts the local HTTPS certificate)
```

Access (ask SD Guthrie IT)

What	Where	Why
Owner on the live site	https://sdguthrie.sharepoint.com/sites/CRS	Admin pages, settings, deploying.
The site's App Catalog	/sites/CRS/AppCatalog	Where the package is uploaded. A site Owner can upload and deploy here. The tenant App Catalog is not used.
Service account gdg@sdguthrie.com	Power Automate (SD Guthrie tenant)	Every flow runs as this account. Needed to edit, import or fix flows.
Term store	/sites/CRS/_layouts/15/termstoremanager.aspx	Business Segment, Department, Unit, Year, Document Type, Confidentiality terms.
A test site set up like CRS	Your own SharePoint site	The code reads and writes real lists (chapter 9).

3. Run it locally

SPFx runs your local code inside a real SharePoint page (the hosted workbench). There is no offline mode, so always point it at a **test site**, never at live CRS.

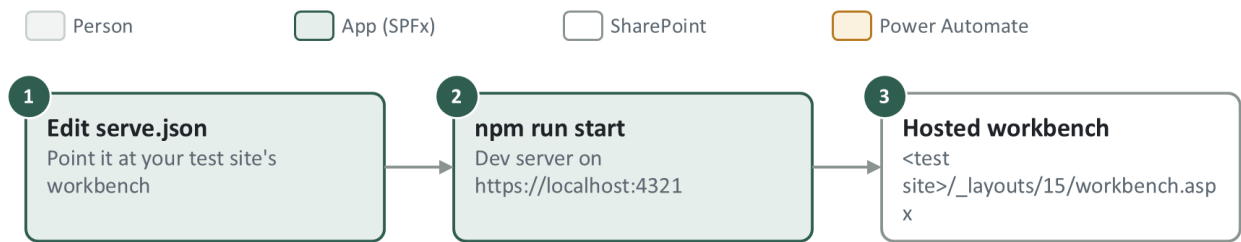


Figure 3. Running locally

1. Open config/serve.json. Change initialPage and serveConfigurations.default.pageUrl to your test site's workbench: `https://<tenant>.sharepoint.com/sites/<test-site>/_layouts/15/workbench.aspx`. At handover they point at the live CRS site.
2. Run `npm run start`. The dev server starts on `https://localhost:4321` and opens the workbench.
3. Add a web part from the toolbox.
 - To test on a real page, open it with `?debug=true&noredir=true&debugManifestsFile=https://localhost:4321/temp/build/manifests.js`.
 - The hideAppBar extension has its own entry in serve.json: set its pageUrl to a library on your test site.
 - Type check: `npx tsc --noEmit`. Tests: `npx heft test --clean`.

Use Heft only. gulp is still listed in package.json, but the build does not use it.

4. Build

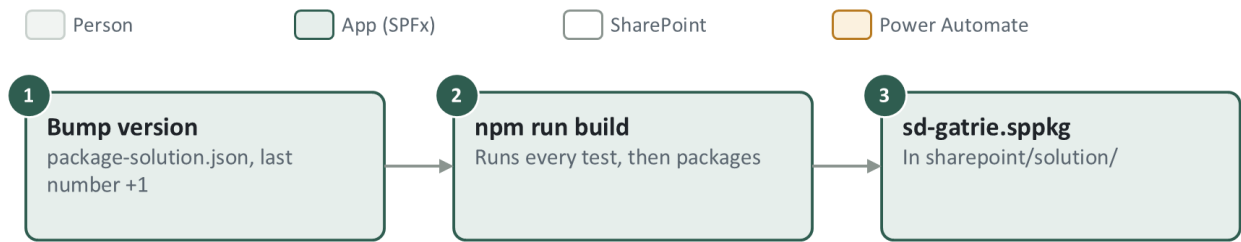


Figure 4. Building the package

1. In `config/package-solution.json`, add 1 to the last number of version (e.g. `1.0.578.0` → `1.0.579.0`). Uploading the same version again makes it look like nothing changed.
2. Run `npm use 22` then `npm run build`. This runs every test, then packages. Output:
`sharepoint/solution/sd-gatrie.sppkg`.
3. Optional checks (Git Bash): the version inside the package, and that your change is really in it.

```
unzip -p sharepoint/solution/sd-gatrie.sppkg AppManifest.xml | grep -o 'Version="[^\"]*"'
unzip -p sharepoint/solution/sd-gatrie.sppkg "ClientSideAssets/<bundle>_*.js" | grep -c "<text
you added>"
```

Always use `npm run build`. Never run `heft package-solution` on its own: it packages the old JavaScript from the previous build.

- A new web part needs its ID in `componentIds` in `config/package-solution.json`, or it will not appear in the toolbox. A test fails the build if it is missing.
- Never change the solution id `4c93b50e-03c1-4005-a280-add0da86c86f`, or SharePoint treats it as a different app.

5. Deploy

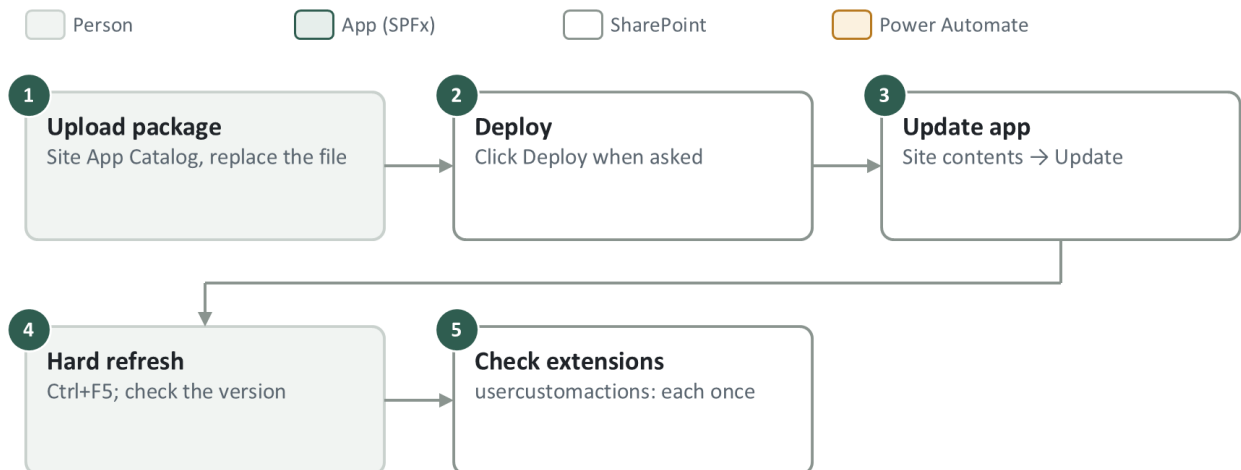


Figure 5. Deploying to the site

1. Open <https://sdguthrie.sharepoint.com/sites/CRS/AppCatalog> (the site's own catalog; the client does not allow the tenant catalog).
 2. Upload `sd-gatrie.sppkg`, replacing the existing file. Click **Deploy**.
 3. Go to **Site contents**, find `sd-gatrie-client-side-solution`, click **Update** if offered.
 4. Hard refresh (Ctrl+F5) or open a new tab. Check the version in Site contents matches the one you built.
- Always deploy to your test site first. The package needs no CDN (`includeClientSideAssets: true`).

Extensions: check they are registered

On the SD Guthrie tenant the package's feature did **not** register the two extensions, so they were added by hand. On a new site, after the first deploy, open

`<site>/_api/web/usercustomactions?$select=Title,Location,ClientSideComponentId`. If one is missing, add it once (two rows load the extension twice) with a POST to `<site>/_api/web/usercustomactions`:

Extension	Title	Location	ClientSideComponentId	Also set
+ New Folder, hides native buttons	CRS New Folder Button	<code>ClientSideExtension.ApplicationCustomizer</code>	<code>c1d2e3f4-a5b6-47c8-d9e0-f1a2b3c4d5e6</code>	—
Approve or reject panel	CRS Bulk Approve	<code>ClientSideExtension.ListViewCommandSet.CommandBar</code>	<code>6d1a9f47-3c82-4e15-b7a0-9f2e5c8d4136</code>	RegistrationId "101", RegistrationType 1

A brand-new component can take several minutes to reach the site after a deploy, with no Update prompt. If the Network tab shows no request for its bundle, wait and refresh.

6. Configuration lists and how the code uses them

Almost all configuration lives in SharePoint lists on the site, not in code. Change a setting in the list and the app picks it up on the next page load (no deploy).

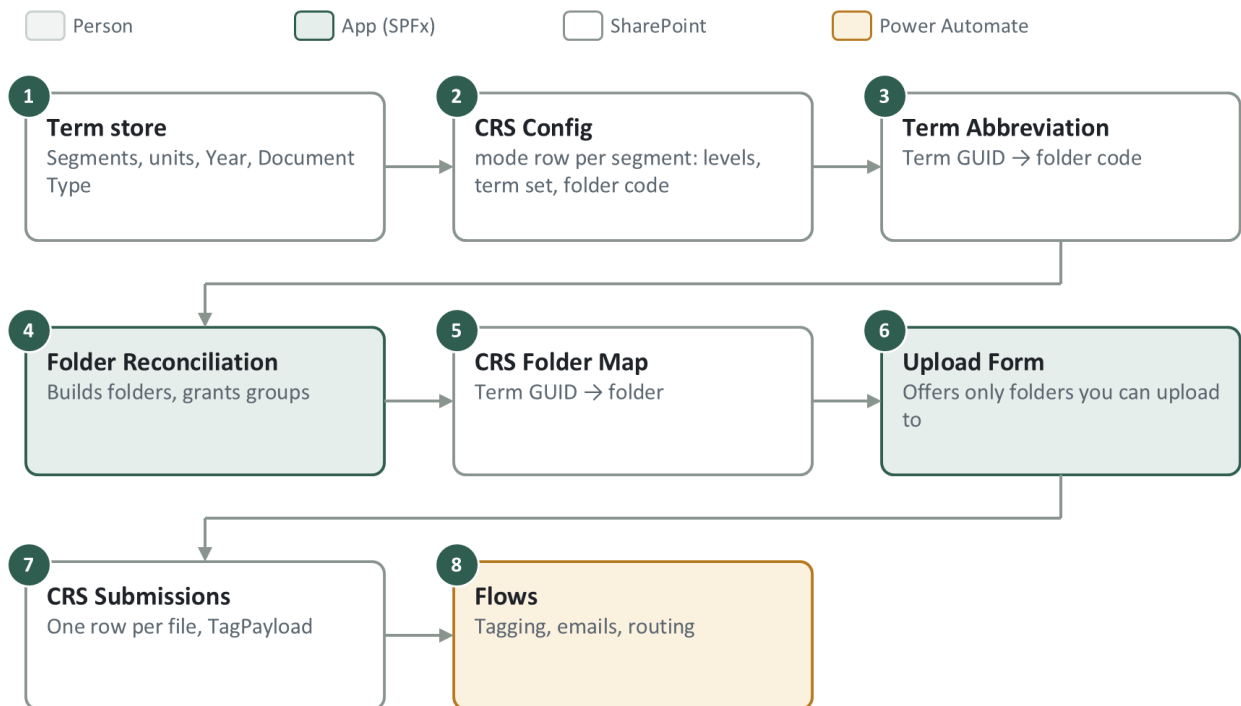


Figure 6. How the configuration connects

How the code finds lists

- src/shared/naming.ts: each list has a fixed suffix (LIST_SUFFIX: Config, Group Map, Folder Map, Term Abbreviation, Submissions, Requests, Audit Log). The code tries the prefix CRS, then the old DMS, and uses the first list that exists.
- Libraries are found from candidate title arrays (LIBRARY_CANDIDATES, DOCUMENTS_CANDIDATES, HC_APPROVAL_CANDIDATES, ...). **A title not in these arrays fails silently.** After the client renames a library, add the new title, build and deploy.
- The hideAppBar extension has its own title list (CRS_LIBRARIES). Add a renamed library there too.
- Use libraryTitle(), documentsLibraryTitle(), libApiTitle() and cachedListTitle(LIST_SUFFIX.x); never a literal title.

The lists

List	What it holds	Code that uses it	Flows that use it
CRS Config	Settings rows, and one mode row per business segment: Levels (JSON of folder tiers), PendingLevels, TermSetGuid, StagingFolder	Nearly every web part. The mode rows drive the Upload Form's choices and folder paths, and Folder Reconciliation.	—
CRS Term Abbreviation	Term GUID → short folder code (e.g. Group IT → GIT)	folderAbbreviation.ts, dmsFolderMap.ts; edited on Folder Administration	—

List	What it holds	Code that uses it	Flows that use it
CRS Group Map	Group ID and name, role (UPL, APR, APRHC, MEMBER...), and the term it applies to	Written by Group Management; read by reconciliation, Upload Form, Search, My Submissions, Requests	NotifyApprovers, reminders, request emails find the approver group here
CRS Folder Map	Term GUID → folder UniqueId and URL	Written by reconciliation; read by <code>dmsFolderMap.ts</code>	—
CRS Submissions	One row per uploaded file (SubmissionFileId, TagPayload, TagStatus, ApproverNotifiedAt...). Survives deletion.	Written by the Upload Form (<code>spSubmissionRecords.ts</code>); read by My Submissions	Apply pending tags, Auto-route, Audit replacements, archive flows, bundled email
CRS Requests	Delete and share requests and their decisions	<code>requests.ts</code> ; Requests page, My Submissions	Notify request activity, Execute approved deletion, Expire shares, Audit request activity
CRS Audit Log	Append-only event log	<code>spAuditLog.ts</code> ; Audit Log page	Audit flows, Auto-route, archive, deletion

CRS Config settings rows

Title	Effect
<code>allowedExtensions</code>	Allowed upload file types (AllowedFileTypes column). Nothing ticked blocks every upload, on purpose. Edited on GDC Configuration.
<code>hcConfidentialityLevel</code>	Which confidentiality term means Highly Confidential. Without it every uploader sees the HC option.
<code>autoApproveOwnUpload</code>	A Head of Unit's own upload approves itself, after a live permission check.
<code>uploadsPaused</code>	Pauses uploads during a folder migration.
<code>allowExternalSharing, tenantDomains</code>	Share requests: whether outside addresses are allowed, and which domains count as internal.
<code>recon_departmentFanOut</code>	Department-level groups reach every unit folder below. On by default; must stay on.

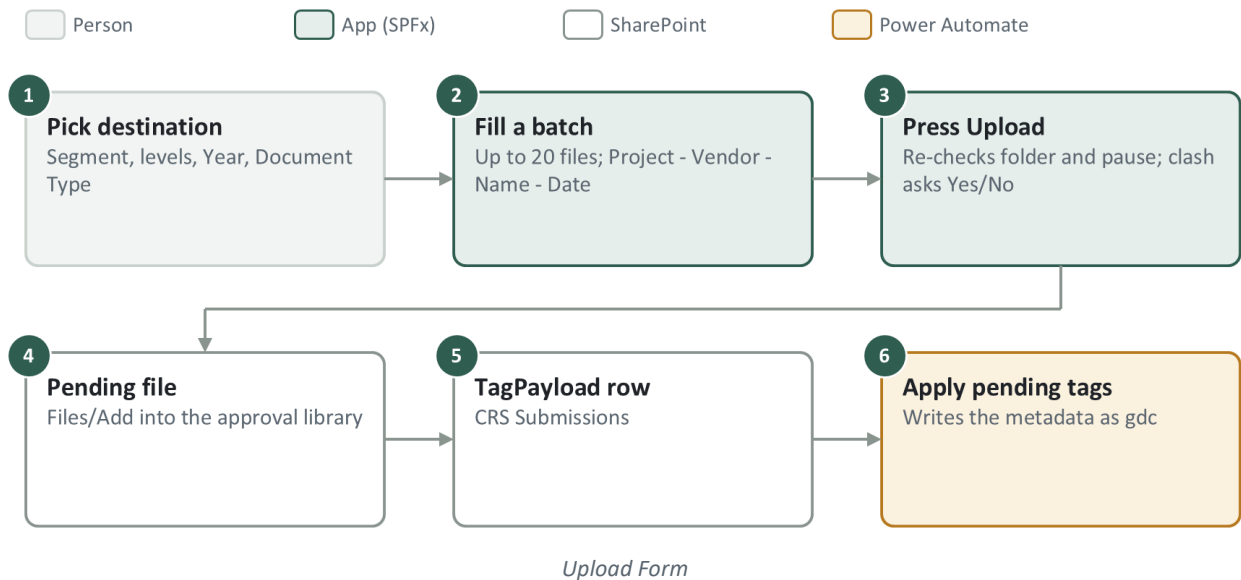
Terms: rename terms, never delete and re-add them. A re-created term gets a new GUID and breaks the rows in Term Abbreviation, Group Map and Folder Map that point at it.

7. Components

One section per web part or extension: a flow diagram, a short explanation, and the files to open first. The numbers in each diagram give the order; the colour says who does the step.

Upload Form

src/webparts/form



The main way documents enter CRS. The uploader picks a destination, fills one batch (one folder, up to 20 files) and presses Upload. Each file lands in the approval library as Pending, and its metadata is written by a flow, because uploaders cannot edit items.

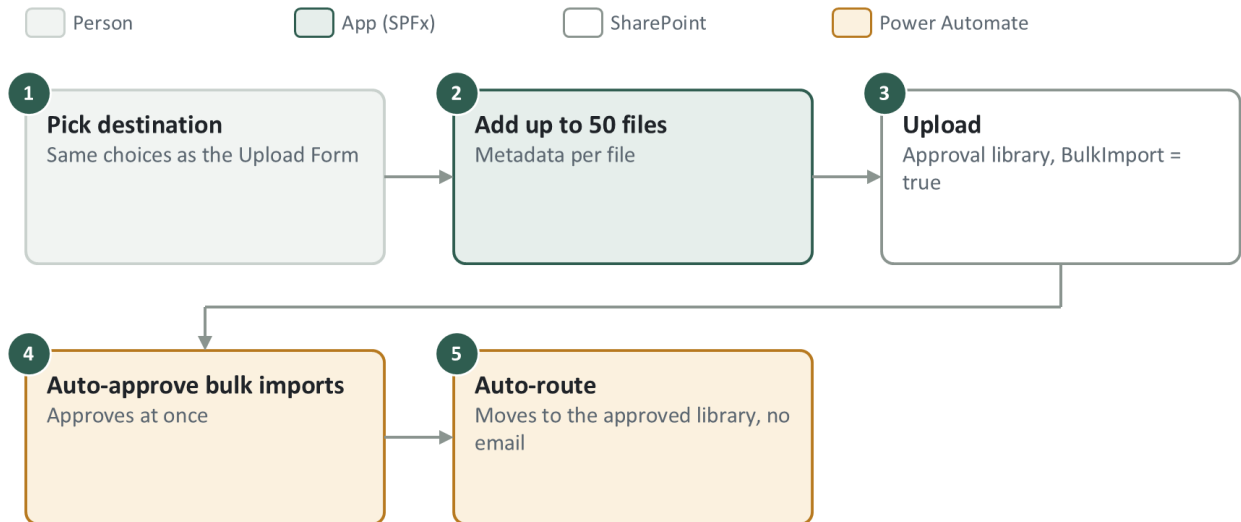
- A folder is offered only if it has a Folder Map row and a permission probe says the user can add items there.
- The destination is fixed when a batch is saved; only one batch is editable at a time. File name = Project - Vendor - Document Name - Date.
- HC is offered only with clearance; the form never creates an HC unit folder.
- A Head of Unit's upload can approve itself (autoApproveOwnUpload) once the tags land.
- A successful batch disappears, a failed one stays: pressing Upload again is a safe retry.

Start reading

- src/webparts/form/components/Form.tsx
- src/shared/formModel.ts
- src/shared/uploadBatches.ts
- src/shared/dmsFolderMap.ts
- src/shared/spSubmissionRecords.ts
- src/shared/clashProxyDeletion.ts

Bulk Upload

src/webparts/bulkUpload



Bulk Upload

For historical documents that were already approved. Files go into the approval library with `BulkImport = true`, a flow approves them straight away, and Auto-route files them.

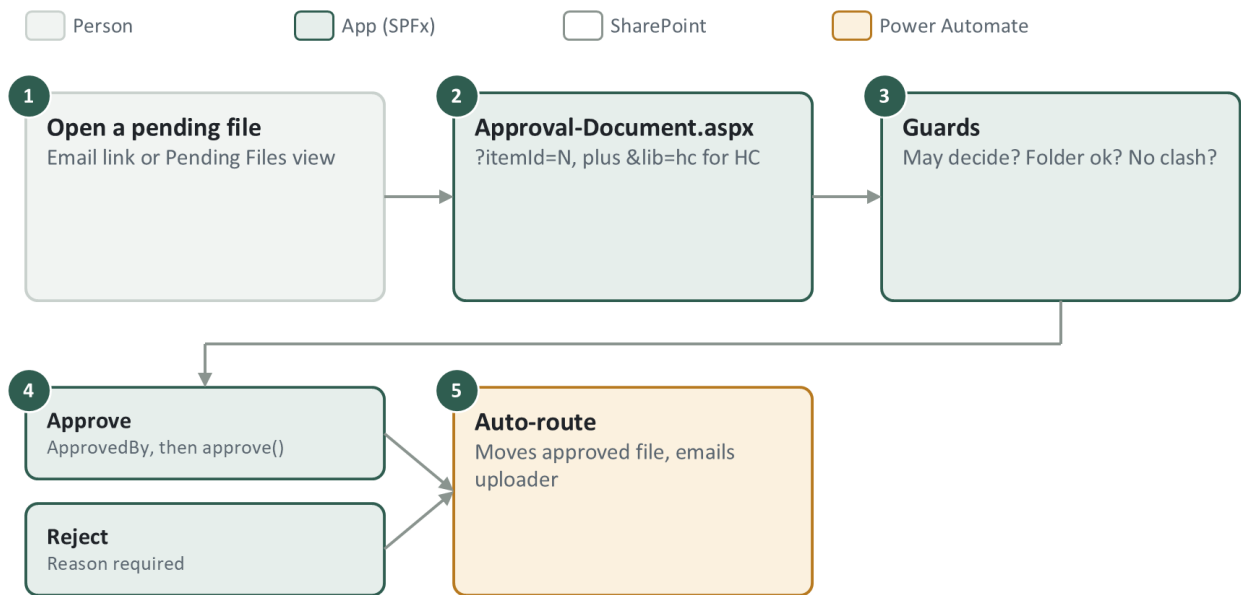
- `BulkImport` must stay visible on both approval libraries, or the flow never sees it.
- Duplicates are checked in both the approval and the approved library.
- HC writes fail closed: they never fall back to the normal library.
- My Submissions shows these as Approved By Guthrie Document Centre, on purpose. No "approved" email is sent.

Start reading

- `src/webparts/bulkUpload/components/BulkUpload.tsx`
- `src/shared/uploadBatches.ts`

Approval Document and the Approve or reject panel

src/webparts/approvalDocument · src/extensions/bulkApprove



Approval Document and the Approve or reject panel

An approver opens one pending file on `Approval-Document.aspx` (or selects many in the library and uses the panel). Shared guards check the decision is allowed, then the page writes the decision and Auto-route does the rest.

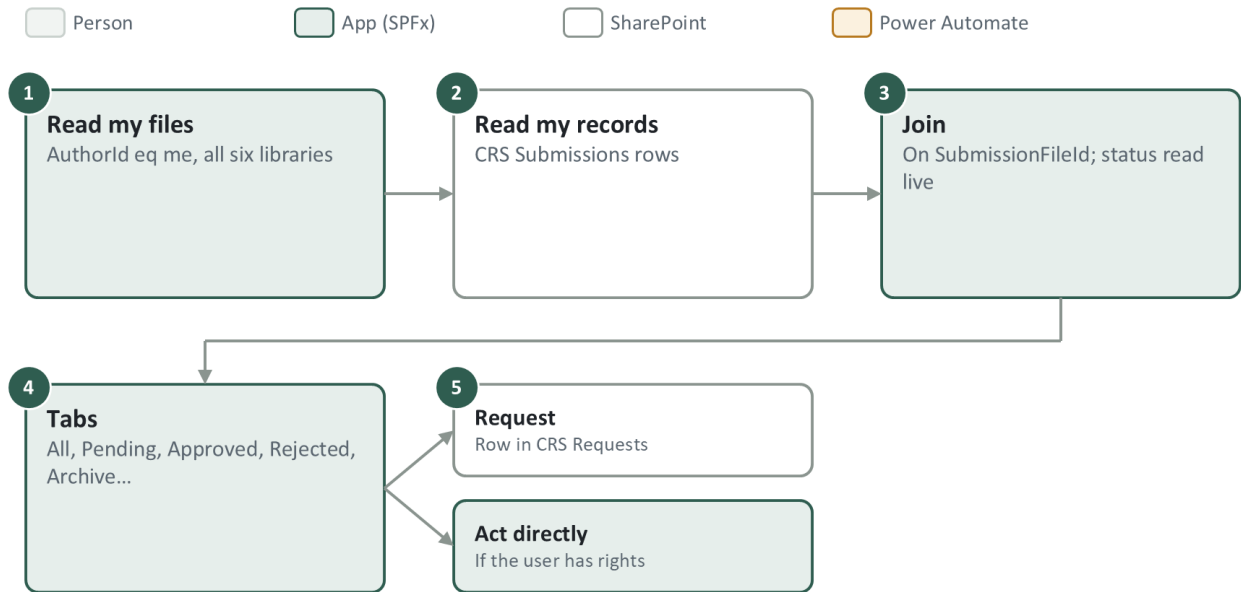
- Never set `odata__ModerationStatus` in the same MERGE as another field: SharePoint returns 500. `ApprovedBy` is written first, then the status.
- The panel uses the same guards as the page (`approvalGuards.ts`); never copy them inline.
- SharePoint's own Approve/Reject is hidden by `hideAppBar` because it skips every guard.
- `?decision=approve|reject` only pre-ticks the choice; it never submits.

Start reading

- `src/webparts/approvalDocument/components/ApprovalDocument.tsx`
- `src/shared/approvalGuards.ts`
- `src/shared/approvalDestination.ts`
- `src/extensions/bulkApprove/components/BulkApprovePanel.tsx`

My Submissions and Document-Viewer

src/webparts/mySubmissions



My Submissions and Document-Viewer

An uploader's own files across all six libraries, joined to their CRS Submissions records so a file stays visible after it is deleted, replaced or archived. From here the user raises delete and share requests, or acts directly if they have the rights.

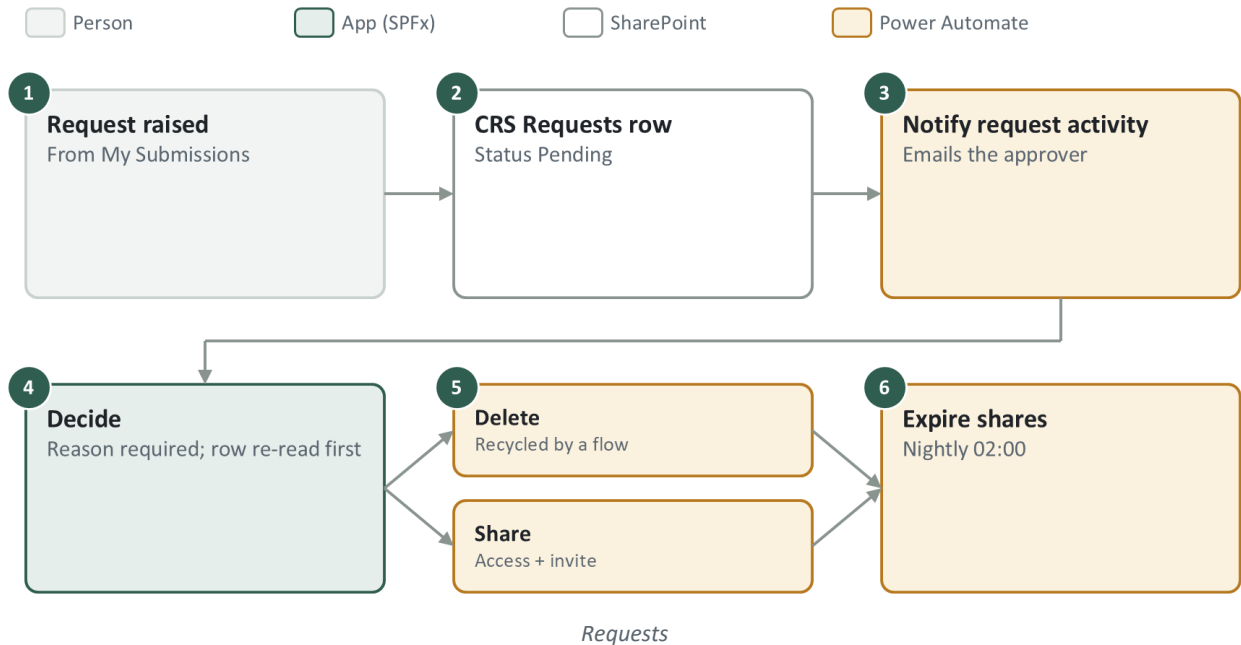
- Status is always read live from the libraries; the record only adds history.
- Document-Viewer.aspx is the same web part in viewer-only mode: ?file=<UniqueId>.
- Reads use \$select fallback steps: one missing column would fail the whole read.
- The file is over 5,000 lines: search for a function by name.

Start reading

- src/webparts/mySubmissions/components/MySubmissions.tsx
- src/shared/mySubmissions.ts
- src/shared/submissionRecords.ts
- src/shared/requests.ts

Requests

src/webparts/requests (page CRS-Request.aspx)



Delete and share requests are rows in CRS Requests. A flow emails the approver, the approver decides on the Requests page, and flows carry out the decision as gdc.

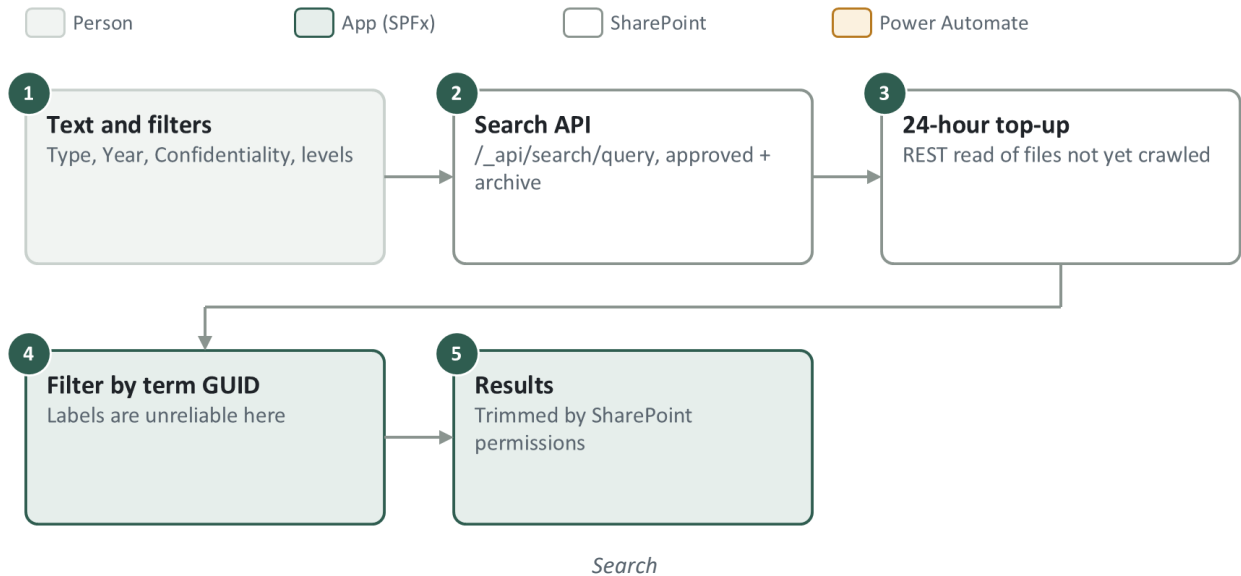
- Who sees which request is `inScope()` in `requests.ts` (approvers: their units; Head of Department: shares).
- If the file was replaced since the request, the request cancels instead of executing.
- The page writes no audit row on decide: the flow does, to avoid duplicates.
- `?request=<Id>` opens one request; `?file=<UniqueId>` gives direct actions from the library.

Start reading

- `src/webparts/requests/components/Requests.tsx`
- `src/shared/requests.ts`
- `src/shared/directFileActions.ts`
- `src/shared/deletionGuard.ts`

Search

src/webparts/documentSearch (home page)



Free text plus filters over the approved and archive libraries, through the SharePoint Search API, topped up with a direct read of the last 24 hours (not yet crawled).

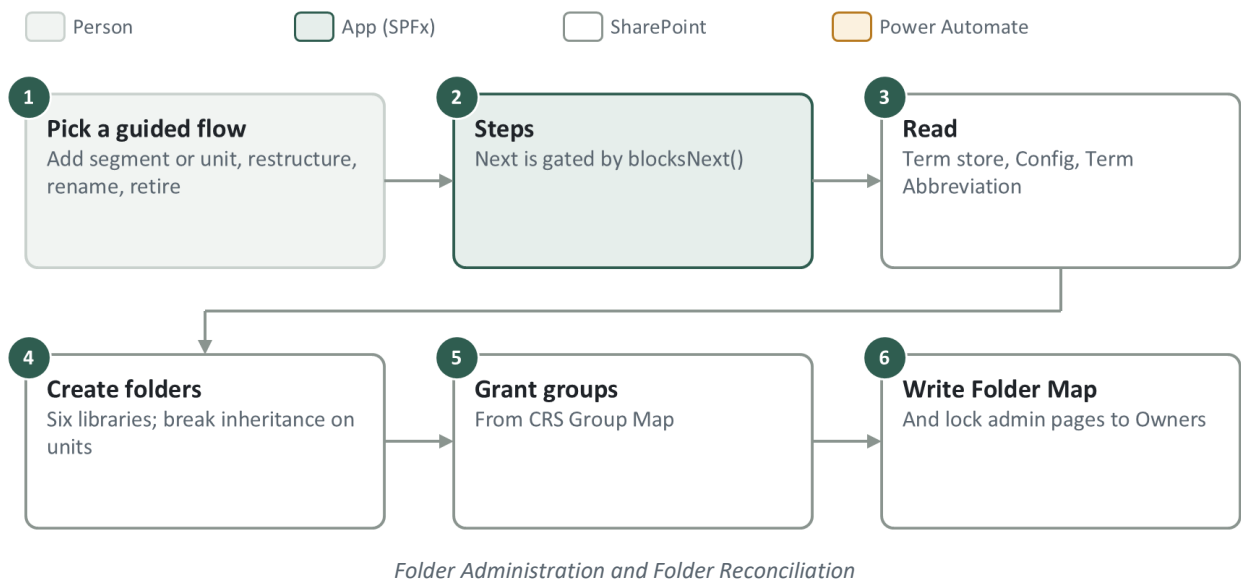
- Approval libraries are not searched (client decision).
- Security comes from SharePoint's permission trimming: never add role checks here.
- Search calls need the header `odata-version: ""` or they fail with 500.
- Stale results: re-index the library (Library settings → Advanced), or check the reindex flow.

Start reading

- src/webparts/documentSearch/components/DocumentSearch.tsx
- src/shared/documentSearch.ts
- src/shared/throttleRetry.ts

Folder Administration and Folder Reconciliation

src/webparts/folderManager (admins only)



Guided flows for adding segments and units, changing structure, renaming and retiring. Folder Reconciliation builds the folder tree in all six libraries from the term store and grants each group its folders.

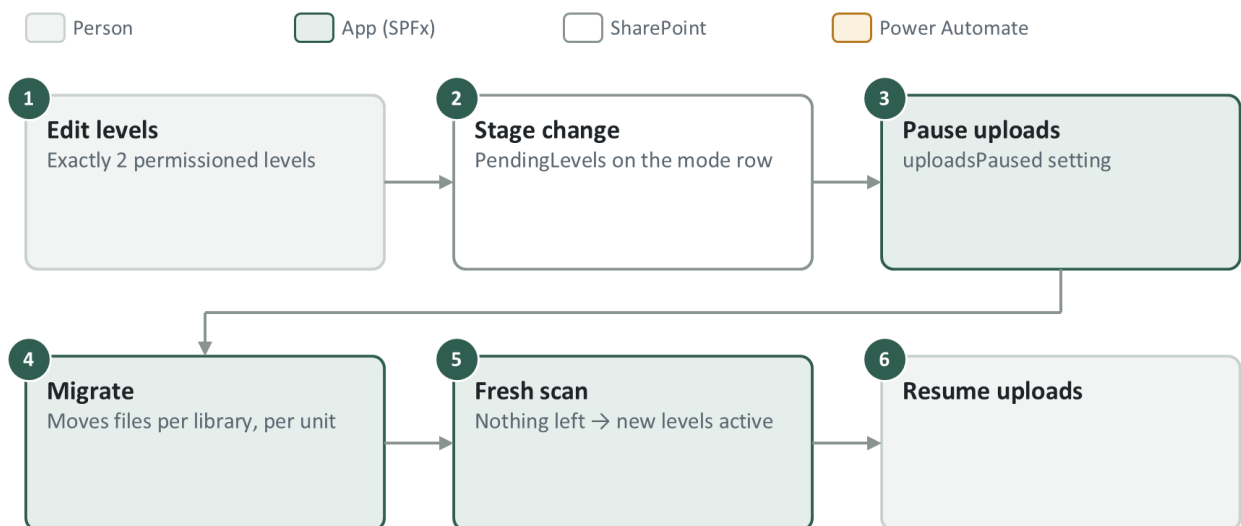
- Reconciliation only adds permissions; it never removes them. It fails closed if it cannot read the permission levels.
- Saving abbreviations only writes rows; reconciliation renames the folders.
- FolderManager.tsx is about 9,000 lines and the most site-tested code: change it narrowly.
- A full run can take tens of minutes on the live site.

Start reading

- src/webparts/folderManager/components/FolderAdmin.tsx
- src/webparts/folderManager/components/FolderManager.tsx (runReconciliation)
- src/webparts/folderManager/components/AbbreviationManager.tsx
- src/shared/folderFlows.ts

Folder Structure and Migration

src/webparts/userAccess



Folder Structure and Migration

Changes a segment's folder levels. On a segment that already holds documents the change is staged, uploads are paused, files are migrated, and the new levels go live only after a clean scan.

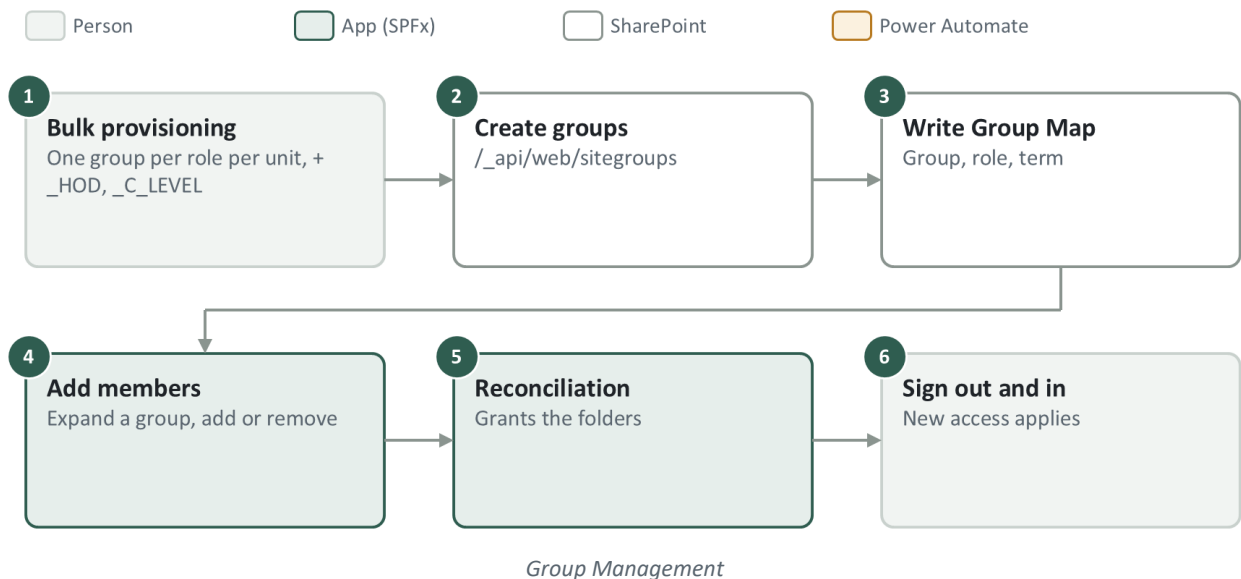
- Exactly two permissioned levels per segment. Year and Document Type are fixed.
- The migrator only walks leaf folders: a file on a non-leaf folder is invisible to it.
- Segments are created, retired and re-coded on the same page (SegmentCreator).

Start reading

- src/webparts/userAccess/components/StructureManager.tsx
- src/webparts/userAccess/components/SubtreeMigrator.tsx
- src/webparts/userAccess/components/SegmentCreator.tsx
- src/shared/subtreeMigration.ts

Group Management

src/webparts/userAccess



Creates the per-unit groups in bulk and their Group Map rows, manages members, and shows the system administrators. Reconciliation then grants the folders.

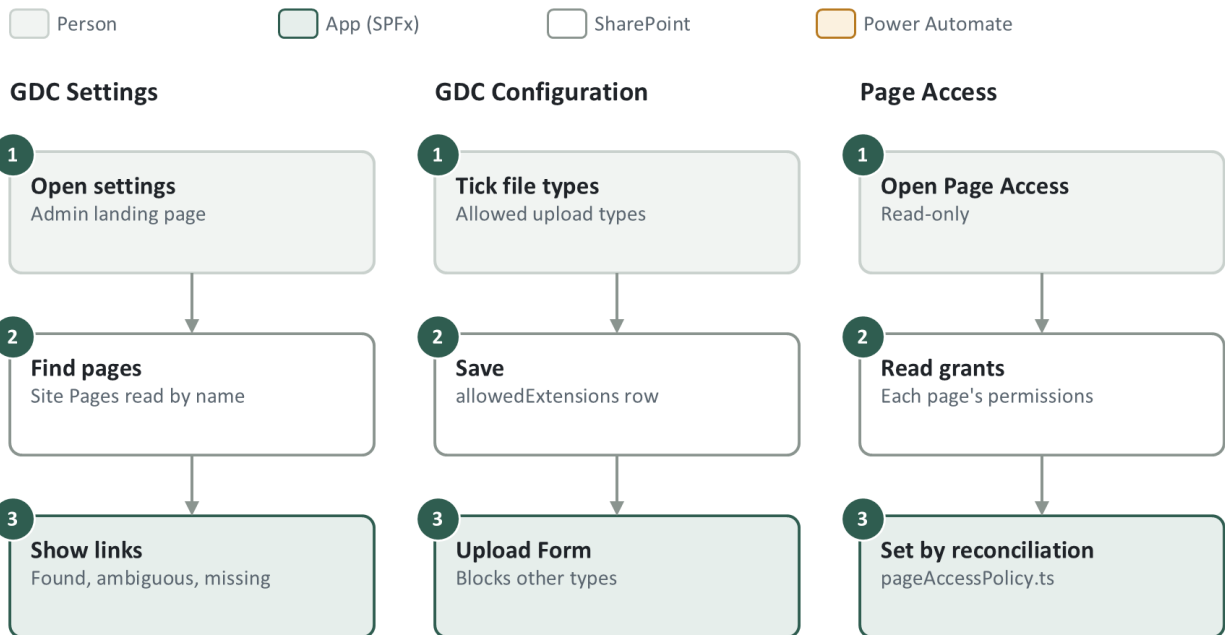
- PERSONAS in `groupMapModel.ts` is the only list of roles.
- Deleting a Group Map row revokes nothing. To remove a role: export members, delete the group, re-create, re-add, reconcile.
- The service account `gdc` is hidden from the administrators list so nobody removes it.
- Members are read with keyset paging (`Id gt last`): `/sitegroups` has no next link.

Start reading

- `src/webparts/userAccess/components/GroupManager.tsx`
- `src/webparts/userAccess/components/BulkGroupProvisioner.tsx`
- `src/shared/groupMapModel.ts`
- `src/shared/spGroups.ts`

Settings, Configuration and Access pages

src/webparts/crsSettings · src/webparts/crsConfiguration · src/webparts/userAccess

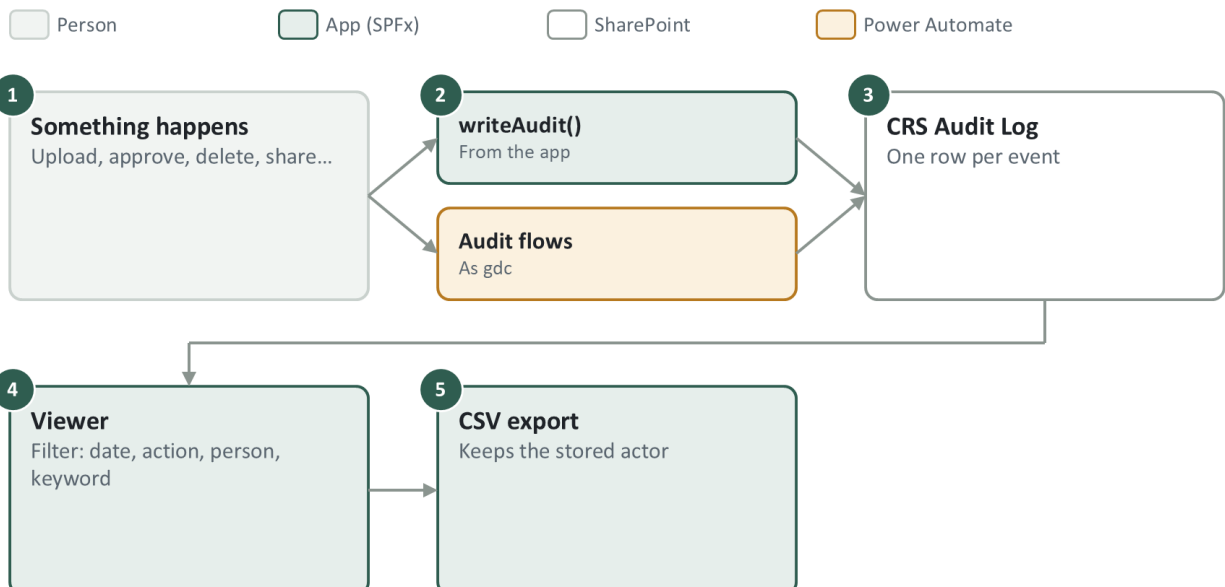


Settings, Configuration and Access pages

- GDC Settings is a directory of admin pages, found by file name at run time because the client renames pages.
- GDC Configuration edits the `allowedExtensions` row. Nothing ticked blocks every upload, on purpose.
- Page Access writes nothing: reconciliation sets every page grant from `pageAccessPolicy.ts`.

Audit Log

src/webparts/auditLog



Audit Log

Every important action writes a row to CRS Audit Log, from the app (`writeAudit()`) or from a flow. The Audit Log page filters and exports them.

- A new event type must be added to `EVENT`, `EVENT_LABEL` and `ALL_EVENT_TYPES`, or it cannot be filtered.
- `writeAudit()` never throws and never blocks the action it records.
- `EventType` is a Text column, never Choice. The total is counted by walking ids (ItemCount is cached and wrong).

Start reading

- `src/webparts/auditLog/components/AuditLog.tsx`
- `src/shared/auditLog.ts`
- `src/shared/spAuditLog.ts`
- `src/shared/displayName.ts`

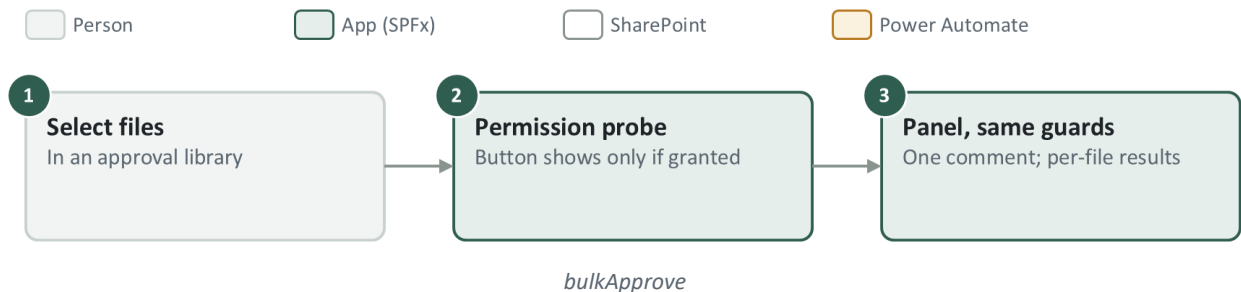
Extensions

`src/extensions`

hideAppBar (application customizer)



bulkApprove (command set)



- On this tenant both had to be registered by hand in `/_api/web/usercustomactions` (chapter 5). Two registrations load the extension twice.
- A renamed library must be added to `CRS_LIBRARIES`, or `hideAppBar` switches itself off there.
- `uploadCommand` is legacy: it links to an old sandbox site and is not registered. Ignore it.

Start reading

- `src/extensions/hideAppBar/`
- `src/extensions/bulkApprove/components/BulkApprovePanel.tsx`
- `sharepoint/assets/elements.xml`

APIs used by each component

Everything uses **SharePoint REST** (`/_api/...`) through `SPHttpClient`, as the signed-in user. No Microsoft Graph. Shared calls live in `src/shared/sp*.ts`. For how to write such a call, see `docs/sharepoint-api-guide.md`.

Component	Lists	Main SharePoint REST calls
Upload Form	Config, Group Map, Folder Map, Term Abbreviation, Submissions, Audit Log	/_api/v2.1/termStore/sets/{id}/children; GetFolderByServerRelativeUrl, folders/AddUsingPath; Files/Add; /items, validateUpdateListItem; EffectiveBasePermissions; approve()
Bulk Upload	Config, Group Map	Term store v2.1; Files/Add; validateUpdateListItem; currentuser
Approval Document	Approval libraries, Config	/items; approve(); MERGE OData__ModerationStatus; file lookups
Approve or reject panel	Approval libraries, Config	MERGE ApprovedBy, then MERGE OData__ModerationStatus (two requests)
My Submissions / Document-Viewer	Six libraries, Submissions, Requests, Group Map, Config	/items with \$select fallback steps; validateUpdateListItem; currentuser
Requests	Requests, Config, Group Map, Audit Log	/items; creates list and columns if missing (/_api/web/lists, CreateFieldAsXml); role assignments; ShareObject
Search	Approved and archive libraries, Config, Group Map	/_api/search/query; term store v2.1; TaxonomyHiddenList
Folder Administration	Config, Group Map, Folder Map, Term Abbreviation, Submissions, Requests	Term store v2.1; folders/AddUsingPath; breakroleinheritance, roleassignments/addroleassignment; sitegroups; creates lists and columns
Group Management, access pages	Config, Group Map, Term Abbreviation, Folder Map	sitegroups (create, members, delete); ensureuser; role assignments; term store v2.1
GDC Settings / Configuration	Site Pages; Config	/items; field choices MERGE
Audit Log	Audit Log	/items; creates the list if missing

REST rules that apply everywhere

- One unknown column in \$select, or one unknown field in a write, fails the **whole** call. Optional columns are read in fallback steps.
- validateUpdateListItem returns HTTP 200 even when a field fails: check HasException. Dates go as M/D/YYYY there, ISO in /items and \$filter.
- Taxonomy values are Label|GUID. Compare terms by GUID: some labels are corrupted on the SD Guthrie tenant.
- Yes/No in \$filter: 1/0, not true/false. Deep paths use GetFolderByServerRelativeUrl(@f)?@f='...'.
- A read capped with \$top cannot prove something is missing: filter server-side or page it. Reads that decide an action are cache-busted.

8. Power Automate

About 25 flows do the work users are not allowed to do themselves. They are **not in the code**. Everything in this chapter was taken from the exports in PowerAutomateFlowsSDG/ (25 September 2026), which are the source of truth: unzip one and read definition.json.

8.1 Where the flows live

What	Detail
Owner	gdc@sdguthrie.com ("Guthrie Document Centre"), the service account. Every flow runs as it.
Environment	SD Guthrie (default) at https://make.powerautomate.com . Sign in as gdc to see them under My flows.
Connections	GDC Proxy - SharePoint (every list and file step) and GDC Proxy - Outlook (every email). Both are signed in with gdc's password.
Exports	PowerAutomateFlowsSDG/*.zip, one per flow. Re-export after every change and commit it.
Run history	Open the flow → 28-day run history. Click a run to see each step's inputs and outputs (Show raw outputs).
Site	Every flow points at https://sdguthrie.sharepoint.com/sites/CRS and at CRS list GUIDs (section 8.6).

8.2 If gdc's password changes: reconnect

When gdc's password changes, its connections break and every flow using them fails (no emails, no tagging, no routing) until they are fixed.

1. Sign in to <https://make.powerautomate.com> as **gdc@sdguthrie.com**.
2. Open **Connections** (left menu, under Data or More).
3. Any connection showing **Reconnect** or **Fix connection** is broken. The two that matter are **GDC Proxy - Outlook** and **GDC Proxy - SharePoint**. Click a connection's name to see which flows use it; one no flow uses can be ignored.
4. Click **Reconnect** and sign in **as gdc** with the new password. Never as yourself: the flows would then act and send email as you.
5. Every flow using that connection is fixed; no flow needs editing. Open one email flow's latest run to confirm it is green.
6. Runs that failed while it was broken do not re-run by themselves. Resubmit them from the run history if the missed emails or actions matter.

8.3 Flow reference

One card per flow (HC twins share a card: they are the same flow pointed at the HC libraries). "Polls" means the SharePoint trigger checks for changes on a timer, so the flow reacts up to that many minutes later.

Routing and tagging

Auto-route approved Pending files to Documents Library (+ HC Auto-route)

Trigger	An item is created or modified in Approval for Document (HC: Approval for Highly Confidential Document). Polls every 5 min. Condition: not a folder.
When approved	Creates the destination folders; if a same-named file is already approved, updates it (new version), otherwise copies the file. Stamps author, editor and dates, copies Document Type, Year, Confidentiality explicitly (a copy drops managed metadata), stamps SubmissionId, BatchId, SubmissionFileId on the copy, marks the old record ReplacedAt/ReplacedBy if it replaced a file, deletes the source, writes Approved and Routed audit rows.
When rejected	Emails the uploader with the approver's comment (OData__ModerationComments).
Emails	"Approved: <file> has been approved" to the uploader (skipped for bulk imports, self-approvals, and uploaders who are site owners). "Rejected: <file> has been rejected" to the uploader.
Watch out	The rejection email says "Rejected By: not recorded" because reject does not write ApprovedBy (known gap, chapter 11). The HC copy must point at HC libraries everywhere.

CRS — Apply pending tags

Trigger	An item is created or modified in CRS Submissions . Polls every 1 min. Condition: TagPayload is set and TagStatus is empty.
What it does	Finds the uploaded file by ItemUniqueId, writes the metadata in TagPayload to it with validateUpdateListItem (as gdc), then sets TagStatus (and TagError on failure) on the record.
Why	Uploaders do not have Edit Items, so they cannot write metadata themselves.
Watch out	If this flow is off, uploads land with no metadata and no approver email (NotifyApprovers waits for the unit tag).

CRS — Auto-approve bulk imports (+ HC auto-approve)

Trigger	Item modified in an approval library. Polls every 5 min. Conditions: not a folder, BulkImport = true, status Pending.
What it does	Sets OData__ModerationStatus to Approved. Auto-route then files it.
Watch out	BulkImport must stay visible on the library; a hidden column is missing from the trigger.

CRS — Approve new folders in Approval Document (+ HC folder approval)

Trigger	New item in an approval library. Polls every 5 min. Condition: it is a folder.
What it does	Approves the folder, so other uploaders can see and navigate it (a pending folder is invisible to them).
Watch out	Must stay on.

Approver emails and reminders

NotifyApprovers (+ HCNotifyApprovers)

Trigger	Item created or modified in an approval library. Polls every 5 min. Conditions: not a folder, not BulkImport, status Pending.
What it does	Reads the file; continues only if UnitTid is set (tagging done) and NextReminderAt is empty. Finds the unit's approver group in CRS Group Map (Role eq 'APR', HC: 'APRHC', matched on UnitTermGuid), reads its members, leaves out the uploader, and sends one email.
Emails	"Approval Requested: <file>" to the unit's approvers. Bcc gdc (the saved copy admins use if someone says they never got it).
Watch out	Site owners and admins who are approvers do get this email (client decision). The flow still contains owner/admin exclusion lookups that do nothing only because their HTTP calls lack an Accept: application/json;odata=nometadata header. Do not add that header to those two lookups: it would switch the exclusion on.

CRS — Reminding Approver to Approve (+ CRS — HC approval reminder)

Trigger	Recurrence, every 30 min.
What it does	Finds Pending files (not folders, not bulk imports) that are due: NextReminderAt has passed, or it is empty and the file is over 3 days old. For each file, finds the approver group (APR / APRHC by UnitTid), leaves out the uploader, gdc and site admins , and emails each approver separately (up to 10). Then moves NextReminderAt 3 days on, even if nobody was emailed.
Emails	"REMINDER: Approval Required for <file>" to each approver.
Watch out	Site admins do not get reminders (kept as is, 30 Sep 2026). The email's link points to ApprovalDocument.aspx; the CRS page is Approval-Document.aspx.

CRS — Notify approvers (bundled) · generated, live on the test site only

Trigger	Recurrence, every 5 min.
What it does	Reads tagged Pending files from both approval libraries (last 3 days) whose CRS Submissions record has no ApproverNotifiedAt. Waits while an upload is still arriving. Groups by upload and unit. HC approvers (APRHC) get all files and both library links; plain approvers (APR) get only the normal files and one link. Then sets ApproverNotifiedAt on each record.
Emails	"Approval Requested: <file>" or "Approval Requested: N documents", in the client's template. Bcc gdc on CRS is still a client decision.
Replaces	NotifyApprovers and HCNotifyApprovers. Switch-over: add the trigger condition @empty(coalesce(triggerOutputs()?['body/SubmissionFileId'], '')) to both, so they only handle old files without a record.
Built by	scripts/build-bundled-approver-flow.py (section 8.6). Needs the column ApproverNotifiedAt on CRS Submissions.

CRS — Remind approvers (bundled) · generated, tested on the test site

Trigger	Recurrence, every 30 min, one run at a time.
What it does	Same due rule as the old reminder. When any file of an upload is due, reminds about all that upload's pending files in one email per audience, and resets NextReminderAt to now + 3 days on all of them (so an upload never splits into two reminders).
Emails	"REMINDER: Approval Required: <file>" or "... N documents", same template and audiences as the bundled approver email. Leaves out uploader, gdc, site admins.
Replaces	The two old reminder flows: turn them off when this one goes on. They share NextReminderAt, so there is no gap or double reminder.
Built by	scripts/build-bundled-reminder-flow.py. Testing helper: scripts/make-reminder-due.js.

Requests (delete and share)

CRS — Notify request activity

Trigger	Item created or modified in CRS Requests . Polls every 5 min.
What it does	Works out the event (new delete or share request, approved, rejected). New request: finds the approver group from CRS Group Map and emails it. Decision: emails the requester (skipped when the requester decided it themselves). Approved share: emails the person it was shared with.
Emails	"Approval Needed to Delete File: <name>" and "File Sharing Request: <name>" to approvers (Bcc gdc). "Approved / Rejected File Deletion Request" and "Approved / Rejected File Sharing Request" to the requester. "A document has been shared with you: <name>" to the recipient.
Watch out	It fires on every edit of a row, so editing an Approved row by hand re-sends its emails. Status Revoked sends nothing.

CRS — Execute approved deletion

Trigger	Item modified in CRS Requests . Polls every 1 min. Acts only when RequestType is Deletion and Status is Approved.
What it does	Checks the file's ETag still matches the request's TargetETag (it was not changed since), recycles the file as gdc, writes a Deleted audit row (once), and updates the request's Status / DecisionNote when it cannot.
Why	Nobody holds a delete permission; every delete goes through this flow.

CRS — Expire shares

Trigger	Recurrence, daily at 02:00 (Singapore time).
What it does	Finds approved Share requests whose ExpiresAt has passed, removes each recipient's access to the file (unless another approved share still covers them), then sets the request to Revoked (RevokedBy = gdc).

Watch out	A blank ExpiresAt means permanent: never touched.
------------------	---

Archive and search

CRS — Archive after seven years (+ CRS — HC Archive after seven years)

Trigger	Recurrence, weekly (Singapore time).
What it does	Finds approved documents created more than 7 years ago, creates the same folder path in the Archive library, moves each file (a move keeps its Uniqueld), restores its fields, resets its permissions to the archive's, stamps ArchivedAt on its CRS Submissions record, and writes an audit row.

CRS — Nightly library reindex

Trigger	Recurrence, every 6 hours (despite the name).
What it does	Changes and restores the NoCrawl setting on the four approved and archive libraries, which makes SharePoint search re-crawl them, so Search results stay current.

Audit

Audit — approval activity (+ HC)

Trigger	Item created or modified in an approval library. Polls every 5 min. Not a folder.
What it does	Writes an Uploaded or Rejected audit row (approvals are written by Auto-route), once per file and event. The actor is the uploader for Uploaded, ApprovedBy (or the editor) for Rejected.

Audit — approval deletions, Documents deletions (+ HC versions: four flows)

Trigger	Item deleted from an approval or approved library. Polls every 5 min. Not a folder.
What it does	Reads the recycle bin for who deleted it and when, and writes a Deleted audit row. The approval-library versions skip files Auto-route just moved (a Routed row in the last 5 min).

CRS — Audit request activity · CRS — Audit replacements

Trigger	Audit request activity: CRS Requests modified, polls every 5 min. Audit replacements: CRS Submissions modified, polls every 5 min, only when ReplacedAt is set.
What it does	Writes request events, and Replaced events, to CRS Audit Log, once each.

8.4 Columns the flows depend on

Renaming, deleting or hiding one of these columns breaks a flow silently. Check this table before changing a list.

Where	Column	Used by (R = reads, W = writes)
Approval libraries	BulkImport	R: NotifyApprovers (trigger), Auto-approve bulk imports (trigger), reminders, Auto-route (email skip). Must stay visible.
Approval libraries	UnitTid	R: NotifyApprovers, reminders, bundled flows (approver group lookup).
Approval libraries	NextReminderAt	W: reminder flows. R: NotifyApprovers, bundled approver email, bundled reminder.
Approval libraries	ApprovedBy	R: Auto-route (approver name, self-approval check), Audit approval activity. Written by the app.
Approval libraries	OData__ModerationStatus, OData__ModerationComments	W: Auto-approve, Approve new folders. R: Auto-route (decision, rejection comment), NotifyApprovers.
Approval libraries	SubmissionId, BatchId, SubmissionFileId	R: Auto-route (copied to the approved file), bundled flows.
Approval libraries	Document_x0020_Type, Year, Confidentiality_x0020_Level	R: Auto-route (copied explicitly), emails.
Approved libraries	SubmissionFileId, Created	R: Archive flows (7-year cutoff, ArchivedAt stamp), Auto-route (replacement).
CRS Submissions	TagPayload, TagStatus, TagError, ItemUniqueId	R/W: Apply pending tags.
CRS Submissions	ReplacedAt, ReplacedBy	W: Auto-route. R: Audit replacements.
CRS Submissions	ArchivedAt	W: Archive flows.
CRS Submissions	ApproverNotifiedAt, SubmissionRef, UploadedAt	R/W: bundled approver email.
CRS Requests	RequestType, Status, ItemUniqueId, RequestedBy, DecidedBy, ShareWith, ExpiresAt, TargetETag, UnitTermGuid, DecisionNote, RevokedBy	Notify request activity, Execute approved deletion, Expire shares, Audit request activity.
CRS Group Map	Role, UnitTermGuid, GroupId	R: NotifyApprovers, reminders, Notify request activity, bundled flows.
CRS Audit Log	Title, EventTime, EventType, Outcome, ActorName, ActorEmail, Source, LibraryName, ItemUniqueId, ItemName, ItemPath, Details, Segment, UnitPath	W: every audit flow, Auto-route, archive flows, Execute approved deletion.

8.5 Emails the system sends

All are sent as gdc through **GDC Proxy - Outlook**. Emails to addresses outside SD Guthrie are blocked by the tenant's mail-flow rule; the bounce lands in gdc's inbox (SD Guthrie IT must add an exception).

Subject	Sent by	To	Bcc
Approval Requested: <file>	NotifyApprovers (+ HC)	Unit approvers (site admins included), not the uploader	gdc
REMINDER: Approval Required for <file>	Reminder flows	Each approver separately (max 10); not uploader, site admins, gdc	—
Approved: <file> has been approved	Auto-route (+ HC)	Uploader (not for bulk imports, self-approvals, owner uploaders)	—
Rejected: <file> has been rejected	Auto-route (+ HC)	Uploader	—
Approval Needed to Delete File: <name>	Notify request activity	Unit approvers	gdc
File Sharing Request: <name>	Notify request activity	Unit approvers	gdc
Approved / Rejected File Deletion Request	Notify request activity	Requester (not if they decided it themselves)	—
Approved / Rejected File Sharing Request	Notify request activity	Requester (not if they decided it themselves)	—
A document has been shared with you: <name>	Notify request activity	The share recipient(s)	—
Approval Requested: <file> / N documents	Bundled approver email (test site)	APRHC / APR audiences	gdc (client deciding)
REMINDER: Approval Required: <file> / N documents	Bundled reminder (test site)	Same audiences, minus site admins	As above

8.6 How-tos

Export and back up a flow

1. Open the flow as gdc → **Export** → **Package (.zip)**.
2. Give it a name, click **Export**, and save the zip into PowerAutomateFlowsSDG/ (replace the old one).
3. Commit it. Do this after **every** change: the export is the only record of what the flow does.

Import or update a flow

1. My flows → **Import** → **Import Package (Legacy)** → choose the zip.
2. For each resource, pick **Create as new** (first time) or **Update** (to overwrite the existing flow).
3. Under Related resources, pick the gdc connections (GDC Proxy - SharePoint / Outlook).
4. Import, open the flow, save, and turn it on.

Point a flow at another site

Each export names CRS's site URL and CRS list GUIDs. Either change every SharePoint action's **Site Address** and **List Name** in the designer, or replace the URL and each GUID in `definition.json` before importing. Get the other site's GUIDs from `<site>/_api/web/lists?$select=Title,Id&$filter=Hidden eq false`. The CRS GUIDs are:

List	CRS list GUID
Approval for Document	eeb1bb19-ec53-4c41-8dc9-ecf255979c9b
Approval for Highly Confidential Document	d935aa6d-dc48-4832-ba7e-eca485ecdff3
Restricted & Confidential Document	fb062dd-fcd0-4458-9161-2bfc19c917fa
Highly Confidential Document	122a4aa9-65fb-479a-90a7-3866d19f51b4
Archive Restricted & Confidential Document	647fd644-ab00-4358-9b3f-49031083fcf0
Archive Highly Confidential Document	fc301054-abf5-4e53-957a-6f8c58a410b8
CRS Submissions	dd1bc5bd-daa7-491a-9dc7-b81212c391ab
CRS Requests	1bcb1aa1-fbf1-4d1b-a6d2-88436824c297
CRS Audit Log	4ac219e5-b855-46a3-be4b-4e96bba546c5

Some actions name lists by title instead (e.g. `getbytitle('CRS Group Map')`); those keep working if the other site uses the same titles.

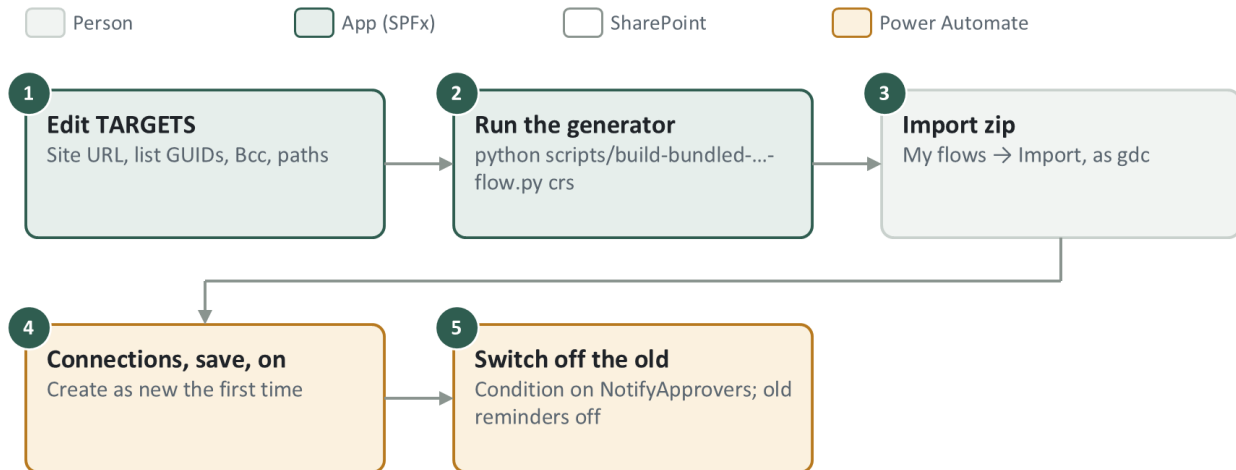
Make the HC copy of a flow

1. Open the normal flow → **Save As**, name it with HC.
2. Change **every** library reference to its HC twin: triggers, Get items, HTTP URIs (`getbytitle(...)` and `lists(guid'...')`), and links in emails. Role APR becomes APRHC.
3. Re-export and search `definition.json` for the normal library names and GUIDs: every HC copy so far has kept at least one. Item IDs are per list, so a missed reference acts on the wrong file.

Read run history and resubmit

1. Open the flow → 28-day run history. Red = failed.
2. Open a run; the failed step is red. Expand it and use **Show raw inputs / outputs** to see the exact request and response.
3. To re-run with the same trigger data, use **Resubmit** on the run.

Rebuild a generated flow



Rebuilding the bundled approver email or reminder

1. Edit TARGETS in `scripts/build-bundled-approver-flow.py` (site URL, list GUIDs, bcc, base export path and output path). The reminder script reuses it and adds its own OUT paths.
2. Run `python scripts/build-bundled-approver-flow.py crs` or `python scripts/build-bundled-reminder-flow.py crs`.
3. Import the zip as gdc (Create as new the first time, Update afterwards), pick the connections, save, turn on.
4. For the approver email: add `ApproverNotifiedAt` (Date and Time, include time) to CRS Submissions first, and the switch-over condition to `NotifyApprovers` and `HCNotifyApprovers` after. For the reminder: turn off the two old reminder flows.

Turn a flow off safely

- A **Recurrence** flow that is off catches up on its next run. A **trigger** flow that is off does not see changes made while it was off.
- Never turn off Apply pending tags, Auto-route or Approve new folders on the live site except for a short, announced window.

8.7 Rules for editing flows

- Build, import and edit flows **signed in as gdc**, and export to `PowerAutomateFlowsSDG/` after any change.
- HTTP header names have no colon (Accept, not `Accept:`). With a colon the header silently does nothing.
- Always send `Accept: application/json;odata=nometadata`, or `?['value']` quietly comes back empty (except the two deliberate `NotifyApprovers` lookups, 8.3).
- Writing a field re-triggers every other "created or modified" flow on that list. Check what else listens first (8.4).
- The connector's Update item action rewrites every column. For one field use an HTTP MERGE.
- Never MERGE `odata__ModerationStatus` together with another field (SharePoint returns 500).
- Type expressions in the **fx** box and never press Enter in it: a pasted line break breaks the expression. A `0` typed into a Condition is text; enter numbers through **fx**.
- A green run does not prove the logic is right. Re-export the flow to confirm an edit landed.
- gdc must stay in the site's **Owners** group. Removing it breaks tagging, notifications and routing at once.

9. Set up a test site

Copy the **structure** of CRS (lists, libraries, columns, views, pages, config rows) with PnP PowerShell, then deploy the app, create the groups and import the flows. No documents are copied. Plan about a day. These steps have not yet been run end to end on a fresh site, so tick each check as you go.

1. **Create the site and get access.** Create a Team site (e.g. /sites/CRS-Test). Add the service account to its Owners. Ask a SharePoint admin for a site App Catalog: `Add-SPOSiteCollectionAppCatalog -Site <test site URL>`.
2. **Terms.** Same tenant: in CRS's term store, open the CRS site collection term group and add the test site under **Site Collection Access**, so it uses the same term sets and GUIDs. Another tenant: `Export-PnPTermGroupToXml / Import-PnPTermGroupFromXml` (keeps term IDs).
3. **Deploy the app first** (chapter 5), then Site contents → New → App → sd-gatrie-client-side-solution. The copied pages contain its web parts. Then register the extensions if missing (chapter 5).
4. **Copy the structure.** PnP PowerShell needs an Entra app registration once per tenant (`Register-PnPEnterIDAppForInteractiveLogin`). Then run the commands below. Leave Group Map and Folder Map empty (the app refills them); do not copy site security.

```
Connect-PnPOnline -Url https://sdguthrie.sharepoint.com/sites/CRS -Interactive -ClientId <app id>
Get-PnPSiteTemplate -Out crs.pnp -Handlers Fields,ContentTypes,Lists,Pages,Navigation -
IncludeAllClientSidePages
Add-PnPDataRowsToSiteTemplate -Path crs.pnp -List "CRS Config"
Add-PnPDataRowsToSiteTemplate -Path crs.pnp -List "CRS Term Abbreviation"

Connect-PnPOnline -Url <test site URL> -Interactive -ClientId <app id>
Invoke-PnPSiteTemplate -Path crs.pnp
```

5. **Check the settings:** content approval On for both approval libraries (drafts visible only to approvers and the author) and Off for both approved libraries; BulkImport visible; NextReminderAt on both approval libraries and ApproverNotifiedAt on CRS Submissions (Date and Time, include time); every page published and named as on CRS.
6. **Groups and folders** (in the app, as Owner): Group Management → Bulk Provisioning; add test users (an uploader and an approver who are not site admins); then Folder Administration → Folder Reconciliation.
7. **Flows:** import them pointed at the test site (8.6). Minimum set: Apply pending tags, Approve new folders (+ HC), NotifyApprovers, Auto-route (+ HC), Auto-approve bulk imports (+ HC).
8. **Smoke test:** upload a file as the uploader (metadata fills in, approver gets an email), approve it (it moves to the approved library within about 5 minutes), check My Submissions.

10. Troubleshooting

Most problems on this project turned out to be a stale browser tab, the wrong site, or a flow. Check in this order:

1. **Which site?** Live and test sites have different list GUIDs, term GUIDs and data.
2. **Which build?** Hard refresh; compare the app version in Site contents with `package-solution.json`.
3. **Real error.** Browser F12: Console, and the Network tab's status and response body.
4. **Is it a flow?** Open the flow's run history as gdc; check the gdc connections (8.2).
5. **Live data.** Run a `scripts/check-*` or `diagnose-*` script in the F12 console.

Symptom	Usual cause
Web part blank, no error	A React hook placed below an early return (React error #310).
A library "disappeared" after a rename	New title missing from <code>src/shared/naming.ts</code> (and <code>CRS_LIBRARIES</code>).
Uploaded file has no metadata	Apply pending tags failed or is off; check the record's <code>TagStatus</code> / <code>TagError</code> .
No approval or reminder email	Flow off or failed; gdc connection broken; the person is the uploader or (reminders) a site admin; or an external address blocked by the tenant.
Uploaders cannot see each other's folders	Folder still pending: Approve new folders must be on.
New member has no access	They must sign out and in; or reconciliation has not run for their group.
Items or groups missing from a screen	A <code>\$top-capped</code> read; filter server-side or page it.
Write returned 200 but nothing changed	<code>validateUpdateListItem</code> reported a field error inside the 200 (<code>HasException</code>).

11. Open items at handover

Ready to roll out on CRS (tested on the test site)

1. Deploy package 1.0.578.0 (CRS runs 1.0.575.0).
2. Bundled approver email: add ApproverNotifiedAt to CRS Submissions; build and import it; then add the switch-over condition to NotifyApprovers and HCNotifyApprovers.
3. Bundled reminder: build and import it; turn off the two old reminder flows; turn the new one on.

Waiting on someone

- Client: keep or drop gdc as Bcc on the bundled emails (bcc in TARGETS["crs"]).
- SD Guthrie IT: a mail-flow exception so flow emails can reach addresses outside SD Guthrie.

Known gaps (accepted for now)

Gap	Effect
Reject does not record who rejected	The rejection email says "Rejected By: not recorded". Fix: write ApprovedBy in its own MERGE before the reject, in ApprovalDocument.tsx and BulkApprovePanel.tsx.
Reminders skip site admins who are approvers	Kept as is (decided 30 Sep 2026). The first approval email does include them.
Notify request activity fires on every edit	Editing an Approved request row by hand re-sends its emails.
Old reminder email's link	Points to ApprovalDocument.aspx; the CRS page is ApprovalDocument.aspx. The bundled reminder fixes this.
Taxonomy labels corrupted on the tenant	Some stored labels show a number. The code compares by GUID.